Reconnaissance des caractères Tifinagh manuscrits par un réseau LeNet-5 implémenté en NumPy

Loubna Zaroual
Master IAA, 2024-2025
Encadré par : Pr. Mohamed Benaddy
Code : https://github.com/[votre-compte]/[votre-depot] (*lien à compléter*)

Résumé

Nous implémentons le réseau convolutif LeNet-5 de LeCun et al. entièrement en NumPy (couches paramétrables, rétropropagation écrite à la main et validée par différences finies, optimiseurs SGD, Momentum, RMSprop et Adam) et l'appliquons à la classification des 33 caractères Tifinagh de la base AMHCD, ramenés à 32×32 pixels en niveaux de gris (25 740 images, découpage 70/15/15). Avec 63 661 paramètres, la version de référence (tanh, pooling moyen, Adam) atteint 98,37 % d'exactitude sur le jeu de test. SGD avec momentum donne 98,45 % ; SGD simple 97,82 %. Remplacer tanh/pooling moyen par ReLU/max-pooling dégrade les résultats (96,66 %) ; ajouter des translations aléatoires et une décroissance du pas les ramène à 98,45 %. Sur un jeu de test de 3 861 images, l'incertitude de mesure est d'environ $\pm 0{,}2$ point : les écarts entre les meilleures configurations ne sont donc pas significatifs.

Mots-clés : LeNet-5, réseau convolutif, Tifinagh, AMHCD, rétropropagation, Adam.

1 Introduction

Le Tifinagh est l'écriture de l'amazighe, langue officielle du Maroc. La reconnaissance automatique de ses caractères manuscrits est un problème de classification d'images à 33 classes, proche de la reconnaissance de chiffres pour laquelle LeNet-5 a été conçu [1]. Ce travail poursuit trois objectifs : (i) implémenter LeNet-5 sans bibliothèque d'apprentissage profond, afin de rendre explicite chaque calcul (propagation avant, gradients, mise à jour) ; (ii) comparer plusieurs algorithmes d'optimisation à architecture constante ; (iii) tester quelques améliorations simples. Le code exécuté est fourni dans le notebook `LeNet5_Tifinagh_executed.ipynb`.

2 Matériel et méthodes

2.1 Données

La base AMHCD (Amazigh Handwritten Character Database) fournie contient 33 classes de 780 images chacune (25 740 images, équilibrées), au format JPEG 64×64. Les dossiers sont nommés par la translittération latine du caractère (`ya`, `yab`, `yach`, …) ; les correspondances avec les glyphes Tifinagh proviennent du fichier d'étiquettes fourni (Fig. 1). L'archive contenait deux copies identiques de la base ; une seule a été utilisée.

Les images sont converties en niveaux de gris, redimensionnées à 32×32 (interpolation bilinéaire), inversées ($x \leftarrow 1 - x/255$: encre à 1, fond à 0) puis standardisées avec la moyenne et l'écart-type du jeu d'apprentissage seul. Le découpage est stratifié : 18 018 images d'apprentissage (70 %), 3 861 de validation (15 %) et 3 861 de test (15 %), avec une graine fixe. Le jeu de test n'est utilisé qu'une fois par modèle, après l'entraînement ; la sélection du « meilleur » modèle s'appuie sur la validation.

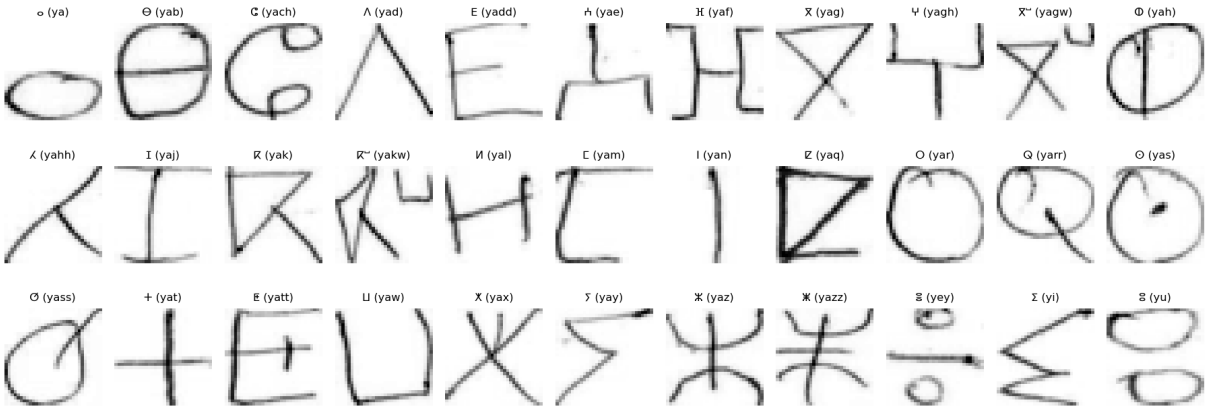


FIG. 1 : Un exemple par classe (glyphe Tifinagh et translittération), après redimensionnement à 32×32.

2.2 Architecture

Le réseau reçoit une image $x \in \mathbb{R}^{1\times32\times32}$ et enchaîne sept couches (Fig. 2). Pour une couche de convolution de noyau $k \times k$, de pas s et de remplissage p, la taille de sortie est

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - k}{s} \right\rfloor + 1. \tag{1}$$

Entrée	C1	S2	C3	S4	C5 / F6	Sortie
1@32×32	6@28×28	6@14×14	16@10×10	16@5×5	$120 \to 84$	33
	conv 5×5	pool 2×2	conv 5×5	pool 2×2	FC	softmax

FIG. 2 : Architecture de LeNet-5 telle qu'implémentée. Chaque convolution et chaque couche FC (hors sortie) est suivie d'une non-linéarité.

TAB. 1 : Couches et nombre de paramètres de la version de référence.

Couche	Sortie	Paramètres	Calcul
C1 (conv 5×5, 6 filtres)	$6 \times 28 \times 28$	156	$6(1 \cdot 25 + 1)$
S2 (pool moyen 2×2)	$6 \times 14 \times 14$	0	–
C3 (conv 5×5, 16 filtres)	$16 \times 10 \times 10$	2 416	$16(6 \cdot 25 + 1)$
S4 (pool moyen 2×2)	$16 \times 5 \times 5$	0	–
C5 (FC, 400→120)	120	48 120	$400 \cdot 120 + 120$
F6 (FC, 120→84)	84	10 164	$120 \cdot 84 + 84$
Sortie (FC, 84→33)	33	2 805	$84 \cdot 33 + 33$
Total		**63 661**	

2.3 Formules de la propagation avant

Convolution. Pour la carte de sortie o d'une couche à C canaux d'entrée,

$$z_{i,j}^{(o)} = b^{(o)} + \sum_{c=1}^{C} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{c,u,v}^{(o)}\, x_{si+u,\, sj+v}^{(c)}. \tag{2}$$

Elle est implémentée par la transformation *im2col* : chaque fenêtre est dépliée en un vecteur de dimension Ck^2, et la couche devient le produit matriciel $Z = \tilde{W}\,\mathrm{col}(x) + b$ avec $\tilde{W} \in \mathbb{R}^{O\times Ck^2}$.

Pooling. Sur chaque fenêtre $\mathcal{R}_{ij}$ de taille 2×2 (pas 2) :

$$y_{ij}^{avg} = \frac{1}{|\mathcal{R}_{ij}|} \sum_{(u,v)\in\mathcal{R}_{ij}} x_{uv}, \qquad y_{ij}^{max} = \max_{(u,v)\in\mathcal{R}_{ij}} x_{uv}. \tag{3}$$

Couche entièrement connectée et activations. $z = W^\top a + b$, avec les activations

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \mathrm{ReLU}(z) = \max(0, z). \tag{4}$$

Sortie et perte. Les $K = 33$ logits z sont convertis en probabilités par le softmax, et la perte est l'entropie croisée moyennée sur un mini-batch de N exemples :

$$p_k = \frac{e^{z_k}}{\sum_{l=1}^{K} e^{z_l}}, \qquad \mathcal{L} = -\frac{1}{N} \sum_{n=1}^{N} \log p_{y_n}^{(n)}. \tag{5}$$

2.4 Rétropropagation

Le gradient de la perte par rapport aux logits est $\partial\mathcal{L}/\partial z^{(n)} = (p^{(n)} - \mathbf{1}_{y_n})/N$. Pour les autres couches, avec $\delta = \partial\mathcal{L}/\partial z$:

$$\text{FC :} \quad \frac{\partial\mathcal{L}}{\partial W} = a\,\delta^\top, \quad \frac{\partial\mathcal{L}}{\partial b} = \delta, \quad \frac{\partial\mathcal{L}}{\partial a} = W\delta, \tag{6}$$

$$\text{Convolution :} \quad \frac{\partial\mathcal{L}}{\partial W_{c,u,v}^{(o)}} = \sum_{i,j} \delta_{ij}^{(o)}\, x_{si+u,sj+v}^{(c)}, \quad \frac{\partial\mathcal{L}}{\partial b^{(o)}} = \sum_{i,j} \delta_{ij}^{(o)}, \tag{7}$$

$$\frac{\partial\mathcal{L}}{\partial x^{(c)}} = \sum_{o} \delta^{(o)} \star \mathrm{rot}_{180}\big(W_c^{(o)}\big) \quad \text{(convolution « pleine »)}, \tag{8}$$

$$\text{Pooling moyen :} \quad \frac{\partial\mathcal{L}}{\partial x_{uv}} = \frac{1}{4}\,\delta_{ij} \quad \text{pour } (u,v) \in \mathcal{R}_{ij}, \tag{9}$$

$$\text{Pooling max :} \quad \frac{\partial\mathcal{L}}{\partial x_{uv}} = \delta_{ij} \text{ si } (u,v) = \arg\max_{\mathcal{R}_{ij}} x, \text{ 0 sinon}, \tag{10}$$

$$\text{Activations :} \quad \tanh'(z) = 1 - \tanh^2(z), \qquad \mathrm{ReLU}'(z) = \mathbf{1}_{z>0}. \tag{11}$$

Ces dérivées ont été implémentées à la main puis validées par différences finies centrées, $\big(\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)\big)/2\epsilon$ avec $\epsilon = 10^{-5}$: l'erreur relative maximale observée dans le notebook est inférieure à 3×10^{-8} pour les quatre combinaisons (tanh/ReLU)×(moyen/max) (vérification sur 4 images, 5 paramètres tirés au hasard par tenseur).

2.5 Initialisation

Les poids suivent l'initialisation de He [3], $W \sim \mathcal{N}\big(0,\, 2/n_{in}\big)$, où $n_{in} = Ck^2$ pour une convolution et le nombre d'entrées pour une couche FC ; les biais sont nuls.

2.6 Algorithmes d'optimisation

Soit $g_t = \nabla_\theta \mathcal{L}_t$ le gradient du mini-batch à l'itération t et η le pas d'apprentissage.

$$\textbf{SGD :} \quad \theta_{t+1} = \theta_t - \eta\, g_t \tag{12}$$

$$\textbf{Momentum :} \quad v_{t+1} = \mu v_t - \eta g_t, \qquad \theta_{t+1} = \theta_t + v_{t+1} \tag{13}$$

$$\textbf{RMSprop :} \quad s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \eta\, \frac{g_t}{\sqrt{s_t} + \varepsilon} \tag{14}$$

$$\textbf{Adam [2] :} \quad m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{15}$$

avec $\mu = 0{,}9$, $\rho = 0{,}9$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ et $\varepsilon = 10^{-8}$.

2.7 Améliorations testées

- **ReLU + max-pooling** en remplacement de tanh + pooling moyen.
- **Augmentation de données** : translation aléatoire de ± 2 pixels (horizontale et verticale) de chaque image du mini-batch.
- **Décroissance du pas** : $\eta_t = \eta_0 \, \gamma^t$ avec $\gamma = 0{,}9$ (t = époque).
- **Dropout** (taux 0,3) sur les deux couches cachées FC : chaque neurone est mis à zéro avec probabilité q et les autres sont multipliés par $1/(1-q)$ à l'apprentissage.

2.8 Métriques

Avec C_{ij} le nombre d'exemples de classe réelle i prédits j (matrice de confusion),

$$\text{Acc} = \frac{\sum_i C_{ii}}{\sum_{i,j} C_{ij}}, \qquad \text{Rappel}_i = \frac{C_{ii}}{\sum_j C_{ij}}, \qquad \text{Précision}_j = \frac{C_{jj}}{\sum_i C_{ij}}. \tag{16}$$

2.9 Protocole expérimental

Toutes les expériences utilisent la même initialisation (graine 0), le même découpage, des mini-batchs de 64 exemples et 15 époques. Les pas d'apprentissage ($\eta = 0{,}05$ pour SGD, 0,01 pour Momentum, 10^{-3} pour RMSprop et Adam) sont des valeurs usuelles, *non optimisées* : aucune recherche d'hyper-paramètres n'a été menée. Chaque configuration n'a été entraînée qu'une fois. L'entraînement s'est fait sur un seul cœur de CPU, soit environ 230 à 260 s pour 15 époques.

3 Résultats

3.1 Modèle de référence

La version fidèle à l'énoncé (tanh, pooling moyen, Adam) atteint une exactitude d'apprentissage de 99,95 % et de validation de 98,26 % ; sur le jeu de test elle obtient **98,37 %** (perte 0,061). Les courbes (Fig. 3) montrent une convergence rapide (plus de 95 % de validation dès la deuxième époque) puis un plateau de la validation autour de 98 % alors que l'exactitude d'apprentissage tend vers 100 % : le réseau sur-apprend légèrement (écart de 1,7 point entre apprentissage et validation).

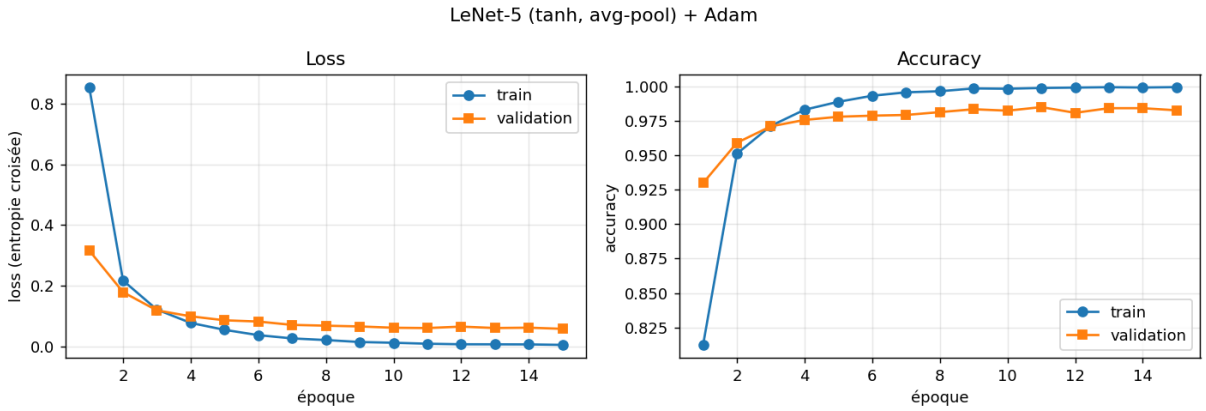


FIG. 3 : Perte et exactitude (apprentissage / validation) du modèle de référence.

La matrice de confusion (Fig. 4) est quasi diagonale. Les erreurs sont rares et isolées (au plus 4 exemples pour une paire de classes). Les plus fréquentes sont ⵣ (yazz)→ⵣ (yaz) (4 cas), ⴹ (yatt)→ⴹ (yadd) (3 cas) et ⴽⵓ (yakw)→ⴽ (yak) (3 cas). Ces paires sont graphiquement voisines : ⴽⵓ est ⴽ avec une labialisation, et ⵣ/ⵣ ne diffèrent que par un trait. Les classes les moins bien reconnues (rappel de 94,9 %) sont ⴽⵓ et ⵣ. Avec 117 exemples de test par classe, un rappel de 94,9 % correspond à 6 erreurs.

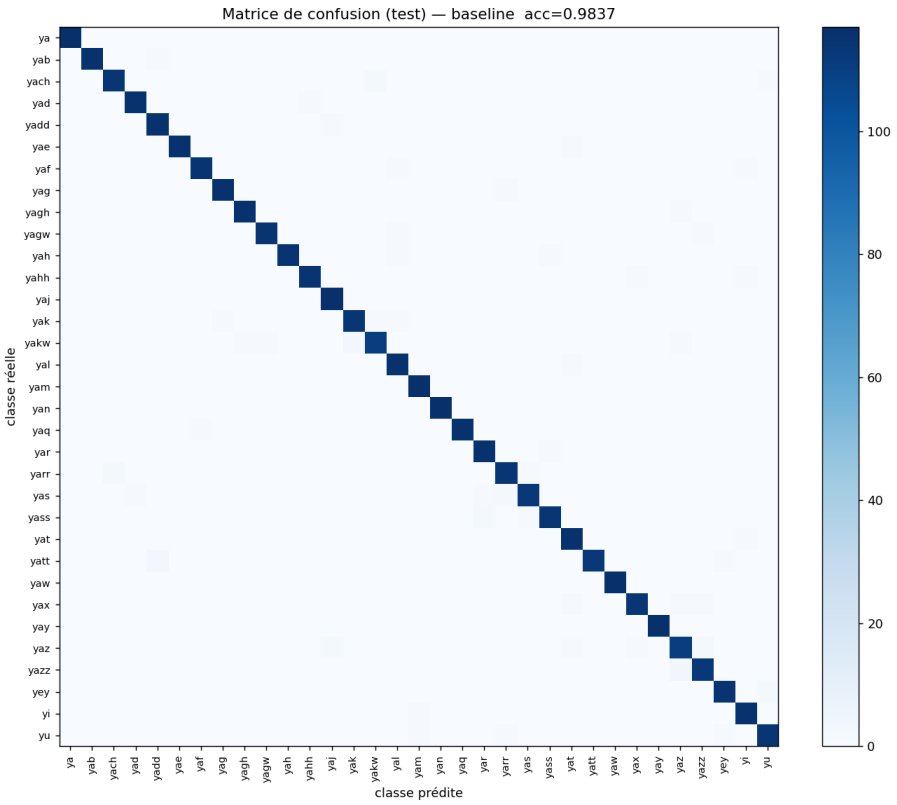


Fig. 4 : Matrice de confusion sur le jeu de test (modèle de référence, exactitude 98,37 %).

3.2 Cartes caractéristiques

La figure 5 montre, pour une image de test, les six filtres 5×5 appris par C1 et les sorties de C1, S2, C3 et S4. Les cartes de C1 réagissent aux bords de l'encre (contours haut/bas, orientations différentes selon le filtre) et reproduisent la forme du caractère à haute résolution. Après S2, la résolution est divisée par deux et les cartes sont plus grossières. Les cartes de C3 et S4 (10×10 puis 5×5) ne sont plus lisibles comme des images du caractère : elles encodent des combinaisons de traits et de positions, ce qui est le comportement attendu d'un réseau convolutif en profondeur. Cette interprétation est qualitative et repose sur l'inspection visuelle d'un seul exemple.

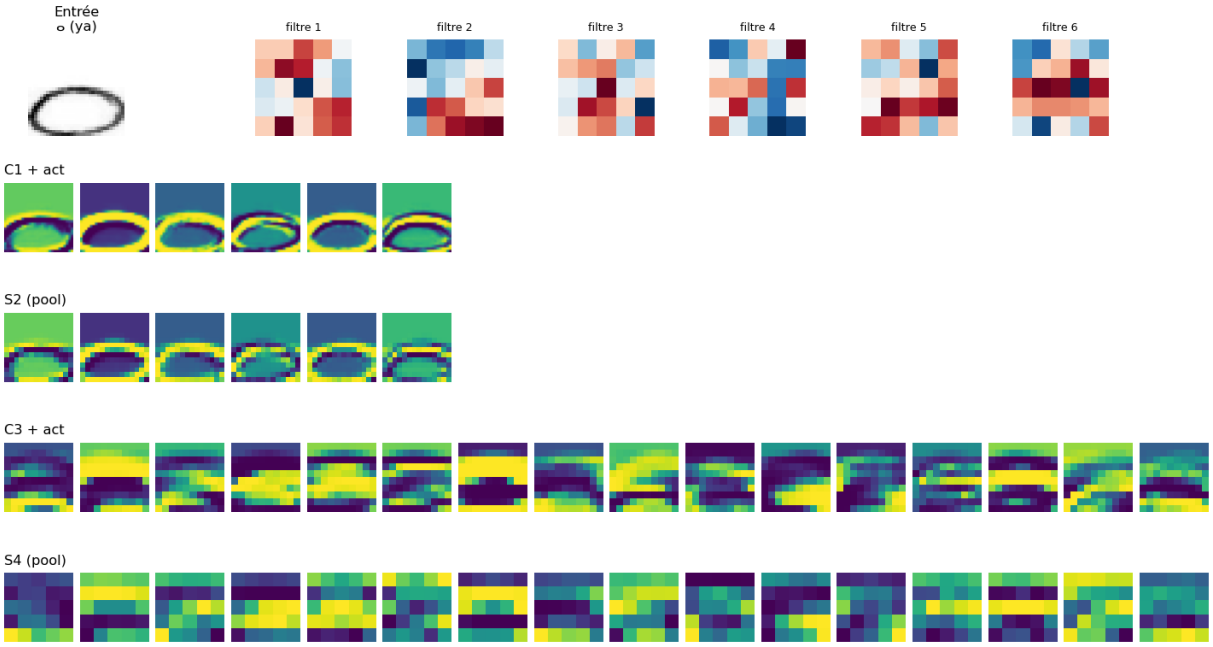


FIG. 5 : Image d'entrée (ɔ, ya), filtres de C1 et cartes caractéristiques de C1, S2, C3 et S4.

3.3 Comparaison des optimiseurs

TAB. 2 : Optimiseurs, même architecture (tanh, pooling moyen), 15 époques. Le test compte 3 861 images.

Optimiseur	Acc. train	Acc. val	Acc. test	Perte test	Temps (s)
SGD ($\eta = 0{,}05$)	99,30	97,80	97,82	0,082	231
SGD + Momentum ($\eta = 0{,}01$, $\mu = 0{,}9$)	99,79	98,29	**98,45**	0,063	229
RMSprop ($\eta = 10^{-3}$)	99,93	98,39	98,08	0,077	232
Adam ($\eta = 10^{-3}$)	99,95	98,26	98,37	**0,061**	232

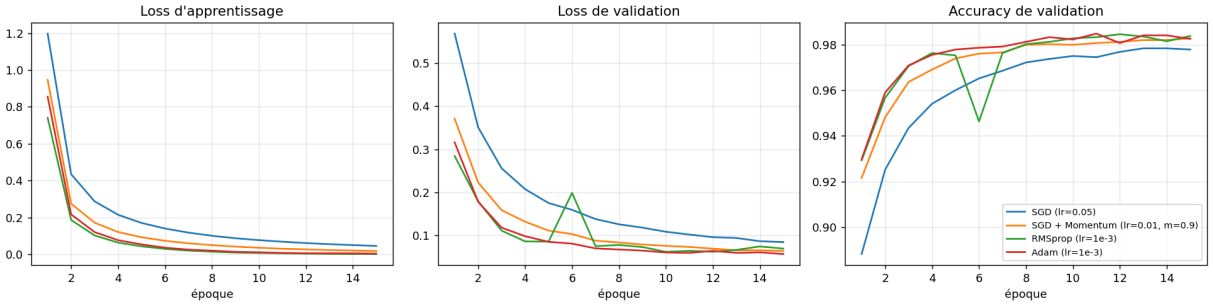


FIG. 6 : Comparaison des optimiseurs : perte d'apprentissage, perte et exactitude de validation.

Le SGD simple est le plus lent à converger et reste le moins précis (97,82 %). Momentum, RMSprop et Adam atteignent des exactitudes de test comprises entre 98,08 et 98,45 %. L'écart-type de mesure d'une exactitude de 98 % sur 3 861 images est $\sqrt{0{,}98 \times 0{,}02/3861} \approx 0{,}22$ point : seul l'écart entre SGD simple et Momentum/Adam dépasse nettement ce seuil, et encore sur une seule exécution par méthode. Nous ne pouvons pas départager Momentum, Adam et RMSprop. Les pas n'ayant pas été réglés individuellement, ces résultats comparent aussi la qualité de nos choix de pas, pas seulement les algorithmes.

3.4 Améliorations

TAB. 3 : Variantes avec Adam ($\eta_0 = 10^{-3}$), 15 époques.

Variante	Acc. train	Acc. val	Acc. test	Perte test
Référence (tanh, moyen)	99,95	98,26	98,37	0,061
ReLU + max-pooling	99,01	96,37	96,66	0,180
ReLU + max-pool + augmentation + décroissance	98,62	**98,63**	**98,45**	**0,058**
+ Dropout 0,3	94,37	98,06	98,03	0,071

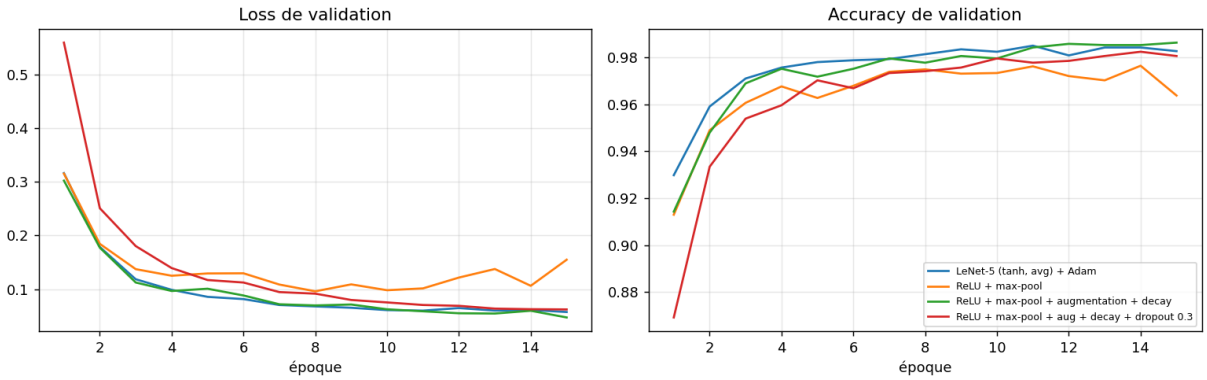


FIG. 7 : Perte et exactitude de validation des variantes.

Le simple passage à ReLU et max-pooling *dégrade* le résultat (96,66 % contre 98,37 %) avec une perte de test trois fois plus élevée (0,180 contre 0,061). L'ajout de translations aléatoires de ± 2 pixels et d'une décroissance du pas corrige cette dégradation et fournit la meilleure exactitude de validation (98,63 %), pour une exactitude de test de 98,45 % et la perte de test la plus basse (0,058). Cette variante est choisie comme « meilleur modèle » sur la base de la validation. Son écart d'apprentissage-validation est nul, ce qui indique que l'augmentation joue bien un rôle de régularisation. En revanche, son exactitude de test est identique à celle de SGD avec momentum et supérieure à la référence de seulement 0,08 point (3 images), ce qui est très inférieur à l'incertitude de mesure. Le Dropout à 0,3 n'apporte pas de gain (98,03 %) : avec 15 époques seulement, le modèle est visiblement sous-entraîné (94,4 % en apprentissage, mesuré avec le dropout actif).

Nous n'avons pas fait d'expérience isolant l'effet de l'augmentation sur l'architecture tanh/moyen, ni celui de la décroissance du pas seule ; nous ne pouvons donc pas attribuer le gain à l'un plutôt qu'à l'autre, ni dire si ReLU/max-pooling est intrinsèquement moins adapté ici ou simplement plus sensible au sur-apprentissage sans régularisation.

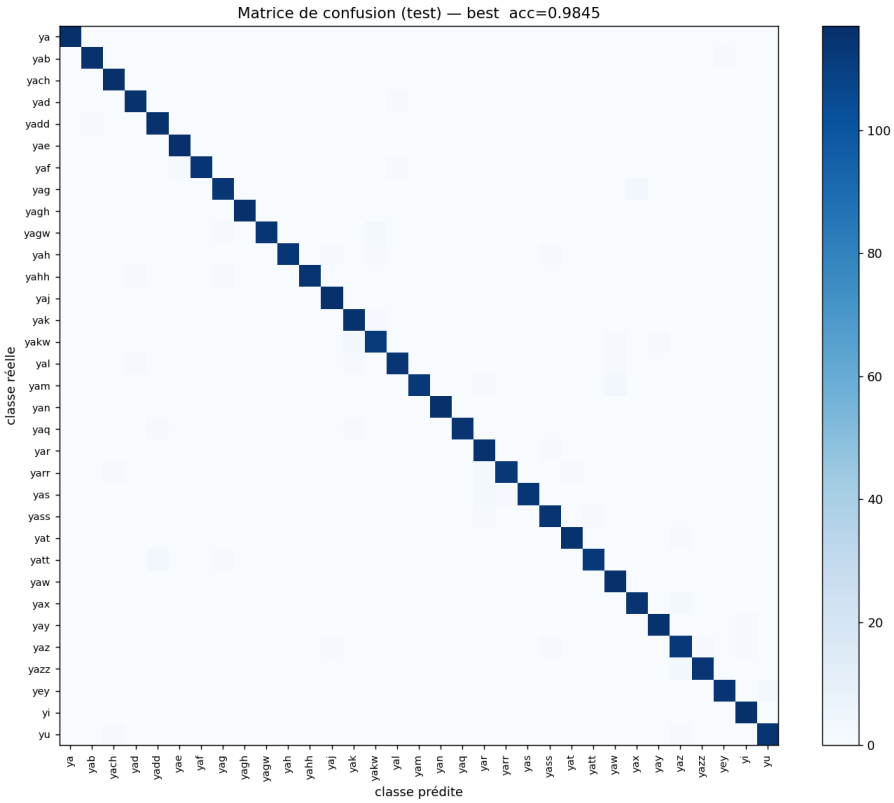


FIG. 8 : Matrice de confusion du meilleur modèle (ReLU, max-pooling, augmentation, décroissance du pas ; test : 98,45 %).

Les confusions du meilleur modèle restent de même nature : ⵣ→ⵣ (3 cas), ⴻ→ⴻ (3), ⴽⵓ→ⴽ (3), ⵛ→ⵍ (3) et ⵅ→ⵅ (3). La classe ⴽⵓ reste la plus difficile (95,7 %). La figure 9 compare les rappels par classe des deux modèles.

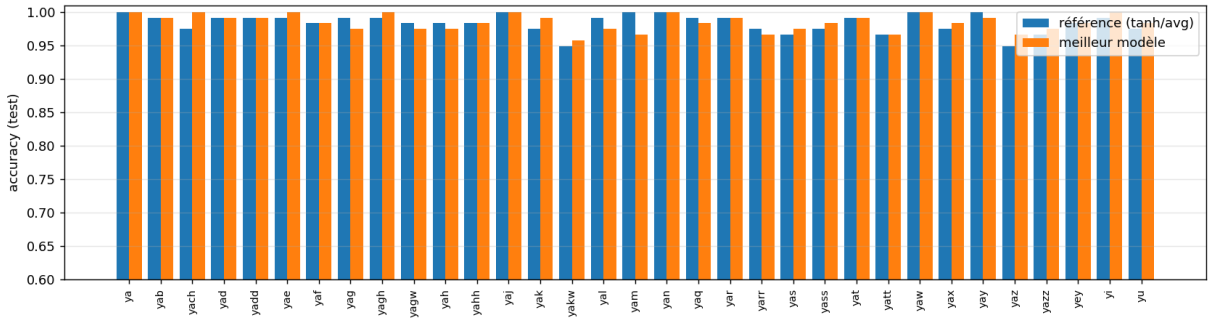


FIG. 9 : Exactitude par classe sur le test : référence et meilleur modèle.

4 Discussion

Ce que montrent les résultats. LeNet-5, avec seulement 63 661 paramètres, atteint environ 98,4 % d'exactitude sur 33 classes de Tifinagh, et les erreurs se concentrent sur des paires de caractères graphiquement proches. Le choix de l'optimiseur influence la vitesse de convergence et, entre SGD simple et les méthodes adaptatives ou à moment, de l'ordre de 0,5 à 0,6 point d'exactitude. Le principal risque de ce réseau est le sur-apprentissage (exactitude d'apprentissage proche de 100 % pour la référence) ; l'augmentation de données par translation est la seule intervention testée qui supprime l'écart entre apprentissage et validation.

Limites.

- Une seule graine et un seul découpage par configuration : aucune variance inter-exécutions n'est estimée. Les écarts inférieurs à environ 0,4 point (deux écarts-types) ne doivent pas être interprétés comme des différences réelles.
- Les pas d'apprentissage n'ont pas été optimisés et 15 époques sont un budget court, ce qui peut avantager ou désavantager certaines méthodes (notamment le dropout).
- La base AMHCD est proche d'un jeu « propre » (caractères isolés et centrés) ; ces performances ne se transposent pas telles quelles à de l'écriture en conditions réelles.
- Redimensionner de 64×64 à 32×32 perd des détails fins, ce qui peut contribuer aux confusions entre caractères proches ; nous n'avons pas testé d'autre résolution.

Écarts par rapport à l'article original. Notre C3 est connectée à toutes les cartes de S2 (2 416 paramètres) et non selon la table de connexion partielle de [1] ; le pooling est un simple moyennage sans coefficient ni biais entraînables ; la sortie est une couche softmax entraînée par entropie croisée et non des unités RBF ; C5 est implémentée comme une couche entièrement connectée sur les 400 valeurs aplaties de S4, ce qui est équivalent à une convolution 5×5 à 120 filtres.

5 Conclusion

Nous avons implémenté LeNet-5 de bout en bout en NumPy, vérifié ses gradients numériquement, et obtenu environ 98,4 % d'exactitude de test sur 33 caractères Tifinagh. SGD simple est légèrement en retrait (97,8 %) ; Momentum, RMSprop et Adam sont indiscernables compte tenu de l'incertitude. ReLU/max-pooling seul est moins bon que la configuration d'origine, mais devient équivalent avec augmentation de données et décroissance du pas. Les pistes naturelles sont l'estimation de la variance sur plusieurs graines, un réglage des pas, des ablations séparées de chaque amélioration, et la validation croisée.

Références

[1] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, « Gradient-Based Learning Applied to Document Recognition », *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, nov. 1998.

[2] D. P. Kingma, J. Ba, « Adam : A Method for Stochastic Optimization », *ICLR*, 2015.

[3] K. He, X. Zhang, S. Ren, J. Sun, « Delving Deep into Rectifiers », *ICCV*, 2015.